

MCHub

Plateforme de référencement de serveurs Minecraft & launcher associé

Cahier des charges — compte-rendu de la phase de cadrage
Document rédigé à partir de la discussion projet menée avec Claude Code
Version provisoire — nom de projet et branding non définitifs

1. Vue d'ensemble du projet

MCHub est une plateforme à deux volets destinée à la communauté Minecraft : un **site web** qui sert de boîte à outils aux propriétaires de serveurs (les **clients**), et une **application Electron** (le launcher) utilisée par les **joueurs** pour découvrir ces serveurs et lancer le jeu avec le bon modpack.

- **Le site** est la source de vérité : c'est là que les clients créent et gèrent leur fiche serveur, leur abonnement et leur configuration.
- **L'application** reste volontairement simple dans sa logique : elle lit les données publiées sur le site, les affiche au joueur, et se charge de télécharger/lancer le jeu et les modpacks.
- Contrainte clé imposée dès le départ : les modpacks doivent obligatoirement passer par **CurseForge**, qui vérifie les packs avant publication — un garde-fou contre les modpacks malveillants.

2. Périmètre fonctionnel

2.1 Site web

- **Comptes clients** : inscription/connexion pour les propriétaires de serveurs.
- **Annuaire public des serveurs** : visible par tout le monde, sans compte requis (bon pour le référencement et l'attractivité du site). Seule la gestion d'une fiche (création/édition) nécessite un compte client.

Fiche serveur — deux niveaux d'information bien distincts :

Champs publics (visibles par tous sur le site)

- Nom, description, image/bannière
- Nombre de joueurs en ligne / capacité — **implémenté** : ping en direct du serveur (protocole officiel « Server List Ping »), résultat mis en cache 60 secondes pour limiter la charge. Un petit point de couleur accompagne le texte (vert si le ping a réussi, gris si le statut est inconnu) — idée du client le 15/09/2026, ajoutée le jour même sur le site et le launcher.
- Modpack CurseForge lié (nom, version) — uniquement pour un serveur moddé. Un serveur vanilla n'a pas besoin de modpack.

Donnée protégée (jamais affichée sur le site)

- **IP / adresse de connexion et version de Minecraft** : décision explicite du porteur de projet — cette information ne doit jamais apparaître sur le site, même pour un serveur 100% vanilla. Elle n'est communiquée que par l'application, après connexion du joueur avec son compte Microsoft. Objectif : garantir que tout joueur, même sur un serveur vanilla, passe par le launcher plutôt que par une connexion directe.

Implication technique : il faut donc deux niveaux d'accès à la donnée serveur — une API publique (sans IP) utilisée par le site, et une API protégée (avec IP) accessible uniquement depuis l'application authentifiée.

Point à confirmer : on part du principe qu'un serveur vanilla n'a pas besoin de lien CurseForge (cette contrainte ne concernant que les serveurs moddés) — à valider avec le client.

2.2 Abonnements & facturation

Plusieurs paliers d'abonnement, du gratuit (fonctionnalités minimales) aux paliers payants qui débloquent progressivement :

- La **personnalisation visuelle** de la fiche serveur (thème, mise en page, plus d'images). Idée ajoutée le 14/09/2026 : le client souhaiterait un éditeur à la « site builder », où il placerait librement les éléments de sa page. Recommandation : partir sur un éditeur **par blocs repositionnables** (bannière, description, galerie, stats, réseaux sociaux...) plutôt qu'un placement 100% libre — le rendu doit rester fiable dans le launcher (Electron) quelle que soit la taille de la fenêtre du joueur, ce qu'un placement totalement libre rend difficile à garantir. À trancher avec le client (voir section 8).
- Le **nombre de serveurs/modpacks** gérables par compte client
- Des **statistiques avancées** (joueurs, téléchargements, etc.), présentées sous forme de **graphiques** (précision du client, pas seulement des compteurs). Deux sources de données distinctes : les **vues de la fiche serveur** sur le site (réalisable dans cette phase), et le **nombre de connexions réelles** au serveur, qui suppose que l'application (Phase 3) existe déjà pour remonter cette information — ne pas promettre les stats de connexion avant que la Phase 3 soit livrée.

Le contenu précis de chaque palier et les montants restent à définir. La facturation elle-même est envisagée via **Stripe** (Billing + Checkout) plutôt qu'un système de paiement maison, pour éviter la gestion de la conformité PCI et sécuriser les paiements récurrents.

2.3 Génération du launcher « brandé »

Un client peut générer, depuis le site, un lien/token de configuration qui, une fois ouvert dans l'application, l'affiche directement sur la page de **son** serveur. Décision retenue : il ne s'agit **pas** d'un exécutable distinct par client (nom/logo personnalisés), mais d'une **seule application**, reconfigurable via ce token — plus simple à maintenir qu'un pipeline de build par client, au prix d'un rendu moins « sur-mesure » visuellement.

2.4 Application Electron (launcher joueur)

- **Connexion Microsoft obligatoire** (à faire) : contrairement au site, aucune donnée n'est accessible dans l'app avant que le joueur se soit connecté avec son compte Microsoft/Xbox Live. Bloqué en attendant l'inscription Azure AD du porteur de projet (voir section 4).
- **Multi-comptes** (à faire) : l'app doit permettre de stocker plusieurs profils/comptes Microsoft localement et de basculer entre eux sans reconnexion complète à chaque fois (utile pour les PC partagés en famille).
- **Liste des serveurs** (squelette fonctionnel construit le 14/09/2026) : lecture des serveurs publiés sur le site via l'API publique (le site reste la seule source de vérité, l'app ne fait qu'afficher). Une première fenêtre Electron existe : elle affiche l'annuaire (bannière, description, type, joueurs en ligne) et une page de détail par serveur, avec un bouton « Rejoindre » volontairement désactivé en attendant la connexion Microsoft.
- **Launcher complet et autonome** (à faire) : télécharge et installe lui-même Minecraft, la bonne version de Forge/Fabric et les mods — comme la CurseForge App ou Prism Launcher. Aucun logiciel tiers requis côté joueur.
- **Bootstrap de l'application** (idée du 14/09/2026, à faire) : prévoir un petit programme de démarrage, séparé de l'application principale, dont le seul rôle est de vérifier au lancement si une nouvelle version du launcher est disponible, de la télécharger/installer si besoin, puis de démarrer

l'application à jour — évite de distribuer manuellement un nouvel installateur à chaque mise à jour, et permet de corriger l'application principale même si elle est cassée, puisque le bootstrap reste indépendant d'elle. Pattern courant chez les launchers tiers (CurseForge App, Prism Launcher, etc.).

*Détail technique du squelette actuel : fenêtre Electron avec processus de rendu isolé (**contextIsolation**, sans accès Node direct) — toutes les requêtes vers le site passent par le processus principal via IPC, jamais directement depuis la fenêtre affichée. C'est la pratique recommandée par Electron pour la sécurité, et elle évite au passage tout problème de CORS.*

2.5 Modération & publication (idée ajoutée le 14/09/2026, pas encore construite)

Avant qu'une fiche serveur soit visible dans l'annuaire public, le porteur de projet veut une étape de vérification pour éviter les publications problématiques. Principe en deux temps :

- **Vérification automatique** : à la création (et éventuellement à la modification) d'une fiche, des contrôles automatiques s'exécutent (mots interdits, doublons suspects, liens/domaines douteux, etc.). Si rien d'anormal n'est détecté, la fiche peut être publiée directement.
- **File d'attente humaine** : si la vérification automatique détecte quelque chose d'inhabituel, la fiche part dans une liste d'attente au lieu d'être publiée — un humain (le porteur de projet ou un modérateur qu'il aura désigné) doit alors l'examiner et l'approuver ou la refuser.
- **Espace d'administration** : un espace séparé du tableau de bord client, réservé au porteur de projet et aux personnes qu'il désigne comme modérateurs, pour traiter cette file d'attente.

*À ne pas confondre avec la « modération participative » du backlog v2 (des joueurs récompensés pour modérer la communauté) — ici il s'agit d'une modération de **publication**, assurée par des personnes de confiance désignées par le porteur de projet, pas par la communauté des joueurs.*

Point de vigilance : la Phase 1, déjà construite, publie une fiche immédiatement à sa création. Ajouter cette étape de vérification plus tard demandera de revenir sur ce comportement (ajout d'un statut « en attente / publié / refusé » sur chaque fiche, et filtrage de ce statut dans l'annuaire public).

3. Modèle de synchronisation

Il n'y a pas de synchronisation bidirectionnelle complexe : **tout se passe côté site** (création/édition des fiches, abonnements, configuration). L'application consomme ces données via une API exposée par le site et se contente de les **afficher correctement** et d'exécuter les actions locales (téléchargement, lancement du jeu). Une simple récupération régulière des données (polling via API REST) est suffisante pour le MVP ; du temps réel (WebSocket) pourra être ajouté plus tard si le besoin apparaît (ex : compteur de joueurs en direct plus réactif).

Implémenté le 14/09/2026 : l'API publique existe concrètement — `/api/public/servers` (liste) et `/api/public/servers/[slug]` (détail), en lecture seule, sans authentification, CORS ouvert (donnée volontairement publique). Elle ne renvoie jamais l'IP, exactement comme les pages du site. C'est cette API que consomme le premier squelette de l'application Electron (voir section 2.4).

Implémenté le 15/09/2026 : le polling régulier lui-même — sur le site, les pages de l'annuaire et de détail se rafraîchissent seules toutes les 30 secondes tant qu'elles restent ouvertes (sans rechargement manuel) ; dans le launcher, la liste des serveurs fait de même en arrière-plan, sans perturber une page de détail en cours de consultation. Repéré par le client en testant : un nouveau serveur ajouté n'apparaissait pas tout seul sur une page déjà ouverte.

4. Contraintes techniques externes & prérequis de comptes

Deux inscriptions externes sont nécessaires, à réaliser par le porteur du projet lui-même :

- **CurseForge / Overwolf** : nécessaire dès la Phase 1 pour lier/vérifier les modpacks (uniquement pour les serveurs moddés). Le porteur de projet possède déjà une clé API CurseForge for Studio — elle sera fournie au moment de coder cette partie de la Phase 1, à examiner ensemble (conditions d'usage commercial notamment). Cette clé devra être stockée de façon sécurisée (fichier d'environnement non versionné), jamais en clair dans le code.
- **Microsoft / Azure** : inscription d'une application Azure AD pour l'authentification Xbox Live / Minecraft Services (flow standard utilisé par des launchers tiers légitimes comme Prism Launcher). Nécessaire à partir de la Phase 3 ; le porteur de projet fera cette inscription lui-même, avec un accompagnement pas à pas au moment venu. Les guidelines Microsoft/Mojang pour launchers tiers devront être respectées (ne pas laisser croire à un launcher officiel, etc.).

5. Stack technique proposée

- **Site web** : Node.js avec Next.js / React — permet de partager du code et des types entre le site et l'application (même écosystème JavaScript/TypeScript).
- **Application** : Electron, sur cette même base JS/TS. Projet initialisé le 14/09/2026 dans le sous-dossier *MCHub/launcher*, séparé du site comme prévu dès le départ.
- **Paielements** : Stripe (Billing + Checkout).
- **Hébergement (site + base de données)** : non tranché. Piste envisagée : Vercel + base managée (Neon/Supabase) pour un démarrage simple et peu coûteux, versus un VPS personnel (OVH, Hetzner...) pour plus de contrôle mais plus de maintenance. Décision à prendre plus tard, sans bloquer le développement local.

6. Direction visuelle & branding

- **Ambiance** : dark mode « gaming », fond sombre avec des accents de couleur vive.
- **Couleur d'accent** : violet / indigo (choix délibéré pour se différencier du vert « Minecraft » utilisé par beaucoup de concurrents).
- **Références retenues** : un compromis entre la densité d'information de la **CurseForge App** et le rendu épuré/moderne de la **Modrinth App**.
- **Cohérence** : le site et l'application doivent partager la même identité visuelle (mêmes couleurs, même typographie, même ton).
- **Nom de projet** : « MCHub » utilisé à titre provisoire dans ce document et pour le dossier de travail — non validé comme nom final, facilement renommable.

Points non tranchés : typographie précise, logo définitif, nom de marque final. À aborder dans une session dédiée au design une fois le périmètre fonctionnel stabilisé.

7. Feuille de route par phases

Phase 1 — MVP

Site web

- Comptes clients (inscription/connexion)
- Création et édition de fiche serveur
- Liaison au modpack CurseForge (serveurs moddés uniquement)
- Annuaire public des serveurs (sans IP : IP réservée à l'application)
- Ping en direct des serveurs (joueurs en ligne / capacité)

Phase 2

Site web

- Système d'abonnement multi-paliers
- Facturation en ligne (Stripe)

Phase 3

Application Electron

- Connexion Microsoft / Xbox Live obligatoire (à faire)
- Gestion multi-comptes/profils joueurs (à faire)
- Liste des serveurs, lecture depuis le site (squelette fait le 14/09/2026)
- Launcher complet autonome (Minecraft + Forge/Fabric + mods) (à faire)
- Bootstrap de mise à jour de l'application (à faire)

Phase 4

Site + Application

- Génération du token de configuration « launcher brandé »
- Affichage automatique de la page du serveur du client via ce token

Phase 5

Site — Administration

- Rôle modérateur/administrateur distinct du compte client
- Statut de publication sur chaque fiche (en attente, publié, refusé)
- Vérification automatique de base (mots interdits, doublons, liens suspects)
- File d'attente + interface de modération pour valider ou refuser les fiches en attente

Backlog / v2 — idées validées, non détaillées

Ces deux idées ont été validées comme pertinentes pour la suite, mais volontairement laissées de côté pour ne pas alourdir le périmètre initial. À redétailler dans une session dédiée une fois les phases 1 à 4 stabilisées :

- **Mise en avant payante de serveurs** (sponsoring) dans l'annuaire et/ou dans l'application.
- **Modération participative** : des joueurs pourraient devenir modérateurs et être récompensés pour cette contribution.

8. Points ouverts à trancher plus tard

- Hébergement définitif du site et de la base de données (Vercel+managed DB vs VPS).

- Prix exact des paliers d'abonnement.
- Détail complet du modèle de données et des rôles/permissions (ex : existe-t-il un rôle admin de la plateforme distinct des comptes clients ?).
- Typographie, logo et nom de marque définitifs.
- Vérification formelle des conditions d'usage commercial de l'API CurseForge.
- Personnalisation visuelle (Phase 2) : éditeur par blocs repositionnables (recommandé) vs placement totalement libre façon « site builder ».
- Modération (Phase 5) : qui peut devenir modérateur et comment (désignation manuelle par le porteur de projet envisagée pour commencer), et quelles règles précises la vérification automatique doit appliquer au départ.

9. Risques & points de vigilance identifiés

- **Dépendance à l'API CurseForge** : c'est un pilier central du produit (vérification des modpacks) ; ses conditions d'usage commercial doivent être validées avant d'investir massivement dans l'intégration.
- **Conformité Microsoft/Mojang** : l'authentification et le launcher doivent suivre les règles d'usage des launchers tiers (inscription Azure, pas de confusion avec un produit officiel Mojang/Microsoft).
- **Sécurité du stockage local** : les tokens des comptes Microsoft multi-profils stockés sur la machine du joueur doivent être chiffrés (ex : coffre du système / stockage chiffré), jamais en clair.
- **Masquer l'IP limite mais ne garantit pas à 100% le passage par le launcher** : cacher l'IP sur le site (et s'assurer que l'API publique de l'annuaire ne l'expose jamais par erreur) empêche la découverte facile — c'est l'objectif recherché, pas une question de confidentialité de l'IP elle-même. Limite à connaître : un joueur qui a déjà rejoint un serveur via le launcher connaît ensuite son IP et peut techniquement s'y reconnecter directement ou la partager — Minecraft ne l'empêche pas nativement. Si ce contournement devient un problème concret, une vérification côté serveur (whitelist liée à la plateforme) pourra être envisagée plus tard.
- **Charge du « ping » en direct des serveurs** : afficher le nombre de joueurs en ligne pour chaque fiche implique d'interroger régulièrement chaque serveur Minecraft listé. **Mitigation déjà en place** : chaque fiche n'est re-pingée qu'au maximum une fois par minute (horodatage *lastPingedAt* en base), le reste du temps la valeur en cache est réutilisée — à revisiter si l'annuaire grossit beaucoup (ping en tâche de fond au lieu d'un ping déclenché par l'affichage de la page).
- **Périmètre ambitieux** : le projet complet (plateforme multi-clients + launcher complet + facturation + branding par client) représente un volume de travail conséquent ; le découpage en phases vise à obtenir une première version utilisable rapidement plutôt que de tout construire d'un bloc.